

NASA Image Search App References & AI Disclosure Document

Date: June 14, 2026

Document Purpose

This document provides a complete record of all external resources consulted and AI tools used during the development of the NASA Image Search App. It is intended to provide full transparency regarding information sources, third-party documentation, and AI-assisted research.

Section 1 — References

The following external resources were consulted directly during development. These include official documentation, library documentation, and personal reference projects.

[1] VS Code React / Node.js Tutorial	
Type	Official Documentation — Microsoft / VS Code
URL	https://code.visualstudio.com/docs/nodejs/reactjs-tutorial
Notes	Used as a refresher on React project setup and command syntax when initializing the app in VS Code.
[2] NASA Image and Video Library API Documentation	
Type	Official API Documentation — NASA
URL	https://images.nasa.gov/docs/images.nasa.gov_api_docs.pdf
Notes	Primary API reference for the NASA Image and Video Library. Used to understand available endpoints, query parameters, response structure, and usage of the api_key.
[3] react-photo-album — Lightbox Integration Example	
Type	Library Documentation — react-photo-album
URL	https://react-photo-album.com/examples/lightbox
Notes	Documentation and examples for integrating react-photo-album with yet-another-react-lightbox. Used to implement the image gallery with full-screen lightbox functionality.
[4] yet-another-react-lightbox	
Type	Library Documentation
URL	https://yet-another-react-lightbox.com/
Notes	Documentation for the lightbox library used alongside react-photo-album to achieve full-screen image viewing.

[5] lightGallery React Library (Evaluated, Not Used)	
Type	Library Documentation — lightgallery.js
URL	https://www.lightgalleryjs.com/docs/react-image-video-gallery/
Notes	Initially evaluated as a gallery solution due to its appealing UI and built-in full-screen support. Not used in the final project — lightGallery requires contacting the maintainers to verify eligibility for a public GNU GPL v3 license, which was not practical for this project.

[6] W3Schools — General Reference	
Type	General Reference — W3Schools
URL	https://www.w3schools.com/
Notes	Used throughout development as a general syntax reference, including CSS properties, Math.floor(), and Math.random().

[7] Game Master App (Personal Project — GitHub)	
Type	Personal Reference Project
URL	https://github.com/cvSquiggles/Game-Master-App
Notes	Developer's own prior project used as a structural reference. Also features a React frontend with a similar API fetch setup, and was referenced for patterns and implementation reminders.

Section 2 — Google AI Search Results Used

The following Google searches returned AI-generated responses (via Google's built-in AI search feature). The relevant portions of those responses are documented below. All responses were from Google's AI search tool and not from a named third-party AI service.

[G1] Google AI Search — VS Code React Snippet Shortcut	
Prompt Summary	"vs code shortcut to get default react view template?"
Response Used	Recommended installing the ES7+ React/Redux/React-Native Snippets extension, then using shorthand such as rfc (React Functional Component), rfce (with default export), or rafc (arrow function component) in a .jsx or .tsx file to generate component templates.

[G2] Google AI Search — React Props & Parent useState	
Prompt Summary	"When you pass a parameter into a react component, does changing the value within that component trigger a state change using the parents useState?"
Response Used	Confirmed that reassigning or mutating a prop inside a child component does not trigger a state change in the parent. React uses one-way data binding (parent to child only). To update parent state from a child, the parent's state setter function must be passed down as a prop and explicitly called within the child.
Notes	This confirmed the expected behaviour and led to the decision to pass both the state value and its setter function as props to the child component.
[G3] Google AI Search — CORS Error on Localhost Fetch	
Prompt Summary	"Access to fetch at https://images-api.nasa.gov/search blocked by CORS policy: Request header field api_key is not allowed..."
Response Used	The response suggested two approaches: (1) remove the api_key parameter entirely, claiming the NASA Images API is public and does not accept an api_key header; and (2) configure a Vite proxy in vite.config.js to forward requests from a local /nasa-api path to the NASA API, masking the localhost origin.
Notes	Note: The claim that the API does not accept an api_key header was incorrect and had already been disproven by the developer via successful Postman testing. The Vite proxy configuration (option 2) was the relevant and useful portion of this response.
[G4] Google AI Search — Converting Search String to Query Parameters	
Prompt Summary	"best javascript method to convert a plain text search string to query params?"
Response Used	Recommended using the native browser URLSearchParams API, which automatically handles formatting and URL-encoding of special characters. Example: new URLSearchParams({ q: textString }).toString() produces a correctly encoded query string.
[G5] Google AI Search — Getting Value from a React Dropdown onChange	
Prompt Summary	"onChange how to get the value of a selected option react drop down element"
Response Used	In React, the selected option value from a dropdown is accessed via event.target.value inside the onChange handler.
[G6] Google AI Search — Ternary Conditional Rendering in JSX	
Prompt Summary	"How to do a ternary in react jsx output to conditionally render component?"
Response Used	Demonstrated the JSX ternary syntax for conditional rendering: wrapping the expression in curly braces within JSX, using the format {condition ? : }.

[G7] Google AI Search — Embedding a For Loop in JSX	
Prompt Summary	"jsx how to embed a for loop"
Response Used	Explained that a traditional for loop cannot be placed directly inside JSX because JSX only accepts JavaScript expressions (code that evaluates to a value), while a for loop is a statement. The standard React approach is to use <code>.map()</code> on an array inline, or to build an array with a for loop before the return block. The response also covered an Immediately Invoked Function Expression (IIFE) as an option for writing a literal for loop directly inside the JSX structure, by wrapping it in an immediately-invoked arrow function that pushes elements to an array and returns it. This was the approach chosen for implementation.

Section 3 — Claude AI Usage Disclosure

The following interactions with Claude (Anthropic) are disclosed below. The model used was Claude Sonnet 4.6 via `claude.ai`.

[C1] Claude (Sonnet 4.6) — React Gallery Component Options	
Prompt Summary	Asked Claude whether React has native library components suitable for displaying a gallery of images from an API, and requested the top two or three options with pros and cons.
Response Used	Claude outlined three options: (1) CSS Grid / Flexbox — full control, no dependencies, but everything must be built manually; (2) <code>react-photo-album</code> — actively maintained, supports rows/columns/masonry layouts, responsive, works with lightbox libraries, TypeScript support; (3) <code>react-image-gallery</code> — polished carousel/slider style with built-in fullscreen and thumbnails, but not ideal for grid layout. Claude recommended <code>react-photo-album</code> paired with <code>yet-another-react-lightbox</code> for an API image display use case.
Notes	This response directly informed the library choice. <code>react-photo-album</code> + <code>yet-another-react-lightbox</code> was ultimately adopted after <code>lightGallery</code> was evaluated and ruled out due to licensing requirements.

[C2] Claude (Sonnet 4.6) — Parsing NASA API JSON Response	
Prompt Summary	Provided Claude with a snippet of the NASA API JSON response and asked how to access the image link and title from within a <code>.map()</code> function. The initial attempt accessed <code>imgData.items</code> directly, which was returning undefined.
Response Used	Claude identified that the NASA API response wraps its results inside a collection object, meaning the items array must be accessed as <code>imgData.collection.items</code> rather than <code>imgData.items</code> directly. Claude also confirmed that within each item, the title is at <code>item.data[0].title</code> and the image URL is at <code>item.links[0].href</code> , with both data and links being arrays requiring explicit <code>[0]</code> indexing. A <code>.filter()</code> guard before <code>.map()</code> was suggested to safely skip any items missing a links array, preventing runtime crashes. The <code>.filter()</code> guard was also implemented.
Notes	The key correction was the collection wrapper — the code was failing silently because <code>imgData.items</code> was undefined. Updating to <code>imgData.collection.items</code> resolved the issue.

[C3] Claude (Sonnet 4.6) — This Document (References & AI Disclosure)	
Prompt Summary	Asked Claude to produce a References and AI Disclosure Document based on project notes documenting all Google searches, Claude interactions, and external resources referenced during development of the NASA Image Search App. Also requested that this prompt itself be included in the disclosure.
Response Used	Claude generated this document (originally as a .docx, then as a .pdf on request), organising all references and AI interactions into clearly labelled sections with consistent formatting. Subsequent update requests consolidated W3Schools references into a single entry [6], removed an unrelated .env entry, revised [C2] to accurately reflect the collection.items correction, and added the for loop JSX entry [G7].
[C4] Claude (Sonnet 4.6) — Project README.md	
Prompt Summary	Uploaded the project zip file (CloudMasonry_DevAssignment.zip) and asked Claude to review the project and generate a README.md file containing the usual information breaking down the application, as well as instructions on how to install the necessary dependencies to run it locally.
Response Used	Claude extracted and reviewed the project source files, then generated a README.md covering: project overview and features, tech stack, project structure with component responsibilities, prerequisites, installation and local run instructions, available npm scripts, API endpoint details, and general project notes.